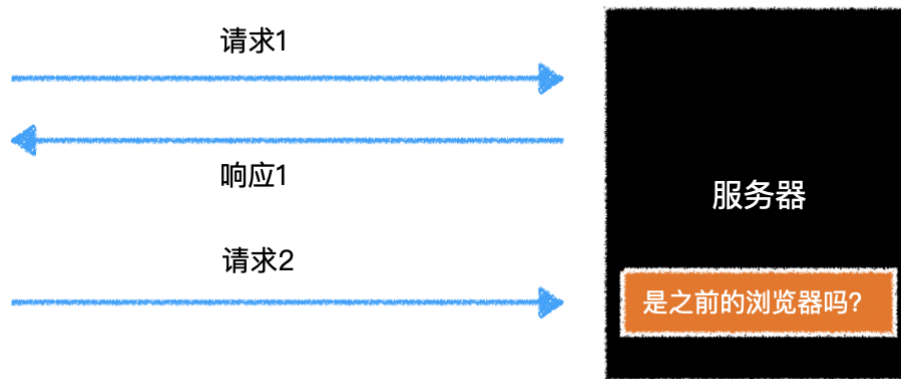


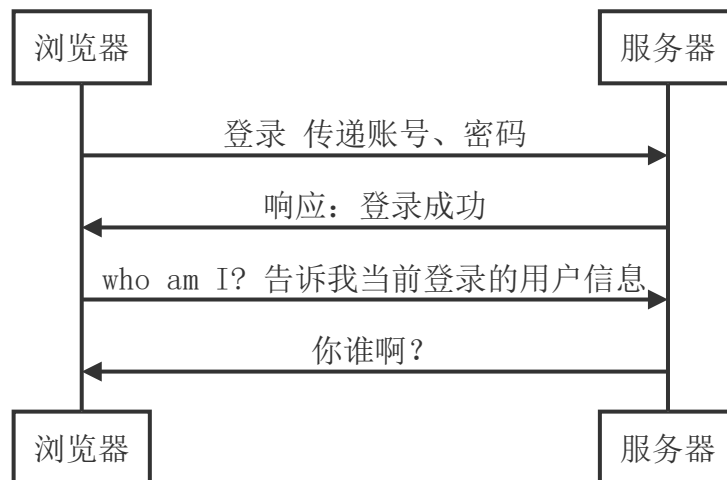
一个问题

思考一个问题，服务器收到了两次请求，根据我们现在学习的知识，服务器如何分辨出两次请求是不是由同一个浏览器发出的。

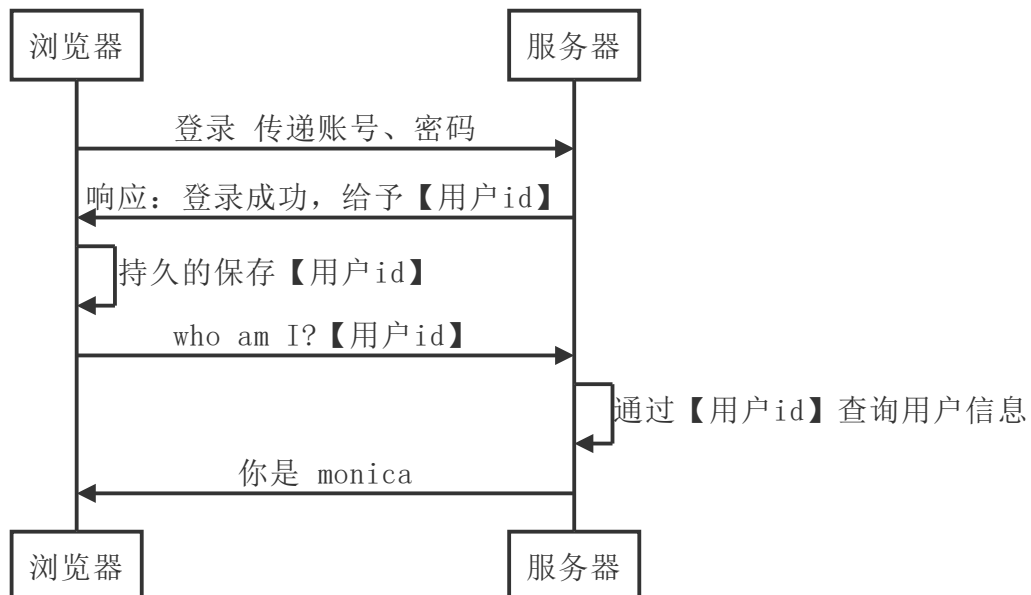


由于服务器特别「健忘」，对于每次请求，服务器都认为是一次全新的请求，用专业的话说，就是服务器**无法保持通信状态**。

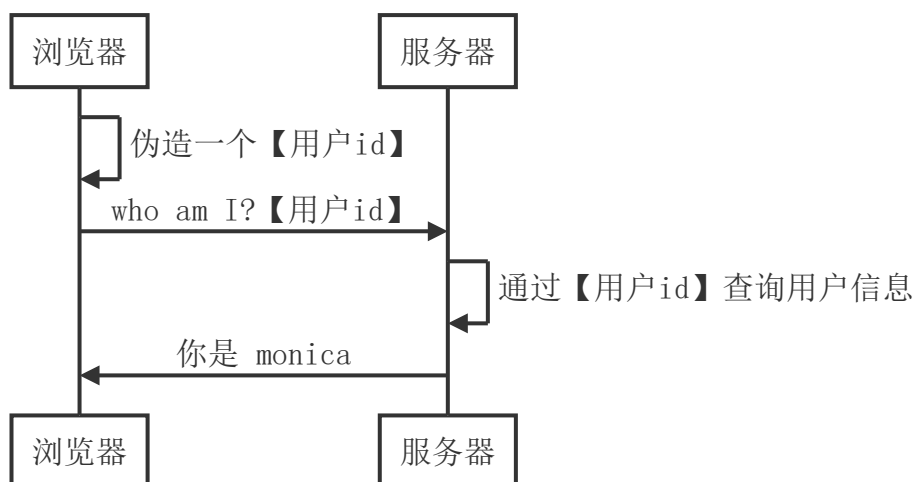
这种问题会集中爆发在登录信息的状态保持上



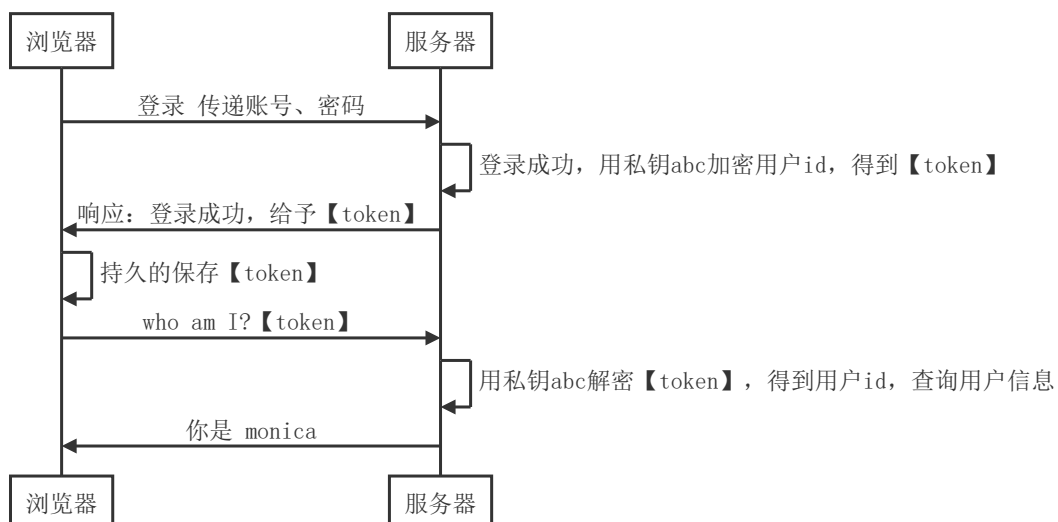
解决思路



这种做法看上去可以解决问题，但是由于用户id明文保存在浏览器，可以被轻易的伪造，用户有机会绕开登录，直接拿到用户信息：



要想给予的状态信息不被伪造，可以把信息在服务器加密后再传递给浏览器：



由于浏览器端的用户无法知道服务器的私钥，因此无法伪造token

cookie

在浏览器端，cookie实际上是一些小文件，可以持久的保存一些数据，通常用它来保存token

浏览器可以保存很多的cookie，每个cookie包含下面的信息：

- key: 键
- value: 值
- domain: 域，表达这个cookie是属于哪个网站的，比如 `duyi.ke.qq.com`，表示这个cookie是属于 `duyi.ke.qq.com` 这个网站的
- path: 路径，表达这个cookie是属于该网站的哪个基路径的。比如 `/news`，表示这个cookie属于 `/news` 这个路径的。
- secure: 是否使用安全传输，跟https协议相关，本文不涉及
- expire: 过期时间，表示该cookie在什么时候过期

设置cookie

当服务器响应浏览器时，可以在消息头中添加一个特殊的字段 `set-cookie`，当浏览器发现响应头中包含 `set-cookie` 时，会自动根据它的值设置 `cookie`

比如，服务器的响应头中包含下面的内容：

```
set-cookie: a=1; domain=localhost; expire=Thu, 29 May 2020 06:33:03 GMT
```

表示设置一个cookie，它的名称是 `a`，值是 `1`，域是 `localhost`，过期时间是 格林威治时间 2020-05-29 06:33:03

设置cookie的完整格式是：

```
键=值; path=?; domain=?; expire=?; max-age=?; secure; httponly
```

- **path**: 设置cookie的路径。如果不设置，浏览器会将其自动设置为当前请求的基路径。比如，浏览器请求的 `path` 是 `/api/user/login`，服务器响应了一个 `set-cookie: a=1`，浏览器会将该cookie的 `path` 设置为请求的路径 `/api/user`
- **domain**: 设置cookie的域。如果不设置，浏览器会自动将其设置为当前的请求域，比如，浏览器请求的地址是 `http://duyi.ke.qq.com/api/user/login`，服务器响应了一个 `set-cookie: a=1`，浏览器会将该cookie的 `domain` 设置为请求的域 `duyi.ke.qq.com`
 - 这里值得注意的是，如果服务器响应了一个无效的域，浏览器是不认的
 - 什么是无效的域？就是响应的域连根域都不一样。比如，浏览器请求的域是 `duyi.ke.qq.com`，服务器响应的cookie是 `set-cookie: a=1; domain=baidu.com`，这样的域浏览器是不认的。
 - 如果浏览器连这样的情况都允许，就意味着张三的服务器，有权利给用户一个cookie，用于访问李四的服务器，这会造成很多安全性的问题
- **expire**: 设置cookie的过期时间。这里必须是一个有效的GMT时间，即格林威治标准时间字符串，比如 `Thu, 29 May 2020 06:33:03 GMT`，表示格林威治时间的 2020-05-29 06:33:03，即北京时间的 2020-05-29 14:33:03。当客户端的时间达到这个时间点后，该 `cookie` 就会过期。
- **max-age**: 设置cookie的相对有效期。expire和max-age通常仅设置一个即可。比如设置 `max-age` 为 `1000`，浏览器在添加cookie时，会自动设置它的 `expire` 为当前时间加上 `1000 秒`，作为过期时间。

- 如果不设置expire，又没有设置max-age，则表示会话结束后过期。
- 对于大部分浏览器而言，关闭所有浏览器窗口意味着会话结束。
- **secure**：设置cookie是否是安全连接。如果设置了该值，则表示该cookie后续只能随着 https 请求发送。如果不设置，则表示该cookie会随着所有请求发送。
- **httponly**：设置cookie是否仅能用于传输。如果设置了该值，表示该cookie仅能用于传输，而不允许在客户端通过 js 获取，这对防止跨站脚本攻击（XSS）会很有用。
 - 关于什么是XSS，不在本文讨论范围

在 express 中，除了可以手动设置响应头的 cookie 外，还可以使用一个 express 中间件 cookie-parser

官网地址：<https://github.com/expressjs/cookie-parser#readme>

用法示例：

```
var express = require('express')
var cookieParser = require('cookie-parser')

var app = express()
// 该中间件将会更改 req 和 res 对象中跟cookie相关的方法和属性
app.use(cookieParser("abc"))

app.get("/test", function(req, res){
  // 在响应头中设置cookie
  res.cookie('键', '值', { // cookie的其他信息
    domain: "域",
    path: "路径", // 默认值为 /
    expires: 过期时间, // Date类型
    maxAge: 相对过期时间, // 单位毫秒，在响应头中会自动转换为秒
    httpOnly: true, // 是否启用 httpOnly
    signed: true, // 启用后，会使用之前配置的私钥，对值进行加密
  })
})
```

发送cookie

浏览器发出请求时，会自动判断是否有满足条件的cookie，如果有，则会将其附带到请求头中
具体的格式如下：

```
cookie: a=1; b=2; c=3
```

上面的示例表示：浏览器找到了3个满足条件的cookie，于是将它们的键值对附带到请求头中
对于是否满足条件的判定如下：

如果一个cookie**同时满足**以下条件，则这个cookie会被附带到请求中

- cookie没有过期
- cookie中的域和这次请求的域是匹配的
 - 比如cookie中的域是 ke.qq.com，则可以匹配的请求域是 duyì.ke.qq.com、ke.qq.com 等等
 - 比如cookie中的域是 duyì.ke.qq.com，则不能匹配 ke.qq.com 这样的请求域

- cookie是不在乎端口的，只要域匹配即可
- cookie中的path和这次请求的path是匹配的
 - 比如cookie中的path是 `/news`，则可以匹配的请求路径可以是 `/news`、`/news/detail`、`/news/a/b/c` 等等，但不能匹配 `/blogs`
 - 如果cookie的path是 `/`，可以想象，能够匹配所有的路径
- 验证cookie的安全传输
 - 如果cookie的secure属性是true，则请求协议必须是 `https`，否则不会发送该cookie
 - 如果cookie的secure属性是false，则请求协议可以是 `http`，也可以是 `https`

服务器获取cookie

`express`可以通过请求头获取cookie

也可以使用下面的代码更加友好的获取cookie

```
req.cookies // 得到解析cookie后的一个对象
req.signedCookies // 得到对加密cookie解密之后的一个对象，解密时使用之前配置的密钥
```

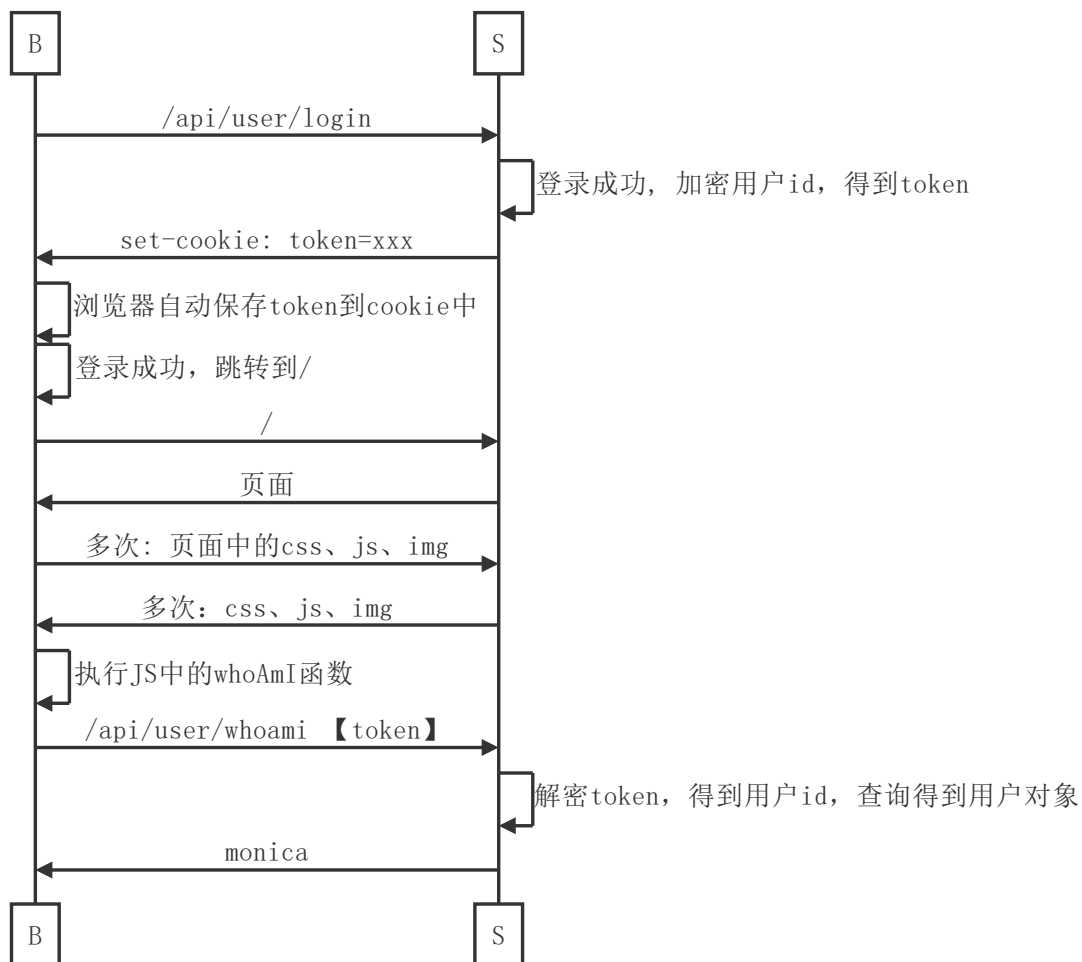
在浏览器端操作cookie

```
document.cookie // 获取所有的cookie
document.cookie = "cookie设置字符串，和响应头中的设置格式一致" // 设置某个cookie
```

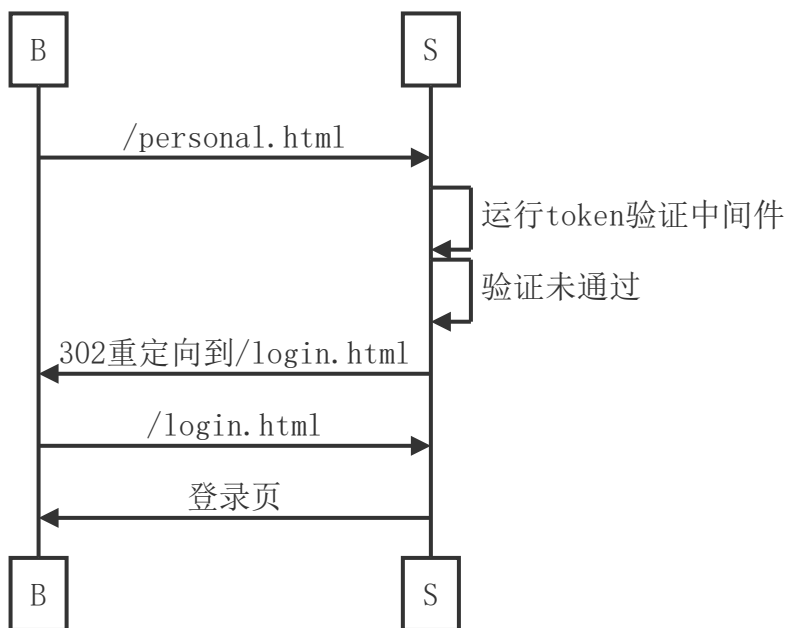
练习

完成 `practice.js` 中剩下的方法

登录完整逻辑



受保护页面访问逻辑【无访问权限】



受保护页面访问逻辑【有权限】

